

Rocketium Playwright Reporter

AWS Deployment Guide

Version: 1.0 | **Date:** June 10, 2026

Repos: playwright-reporter-backend · playwright-reporter-ui · automation-tests-2.0 · rocketium-playwright-reporter

Table of Contents

- 1. Overview
- 2. Architecture Diagram
- 3. What Each AWS Service Does
- 4. Step 1 — RDS PostgreSQL
- 5. Step 2 — S3 Bucket (Artifacts)
- 6. Step 3 — EC2 Backend
- 7. Step 4 — Frontend (S3 + CloudFront)
- 8. Step 5 — Connect Code & Credentials
- 9. Step 6 — Verify End-to-End
- 10. AWS Topology Diagram
- 11. Environment Variables Reference
- 12. Troubleshooting
- 13. Security Groups & Creation Order

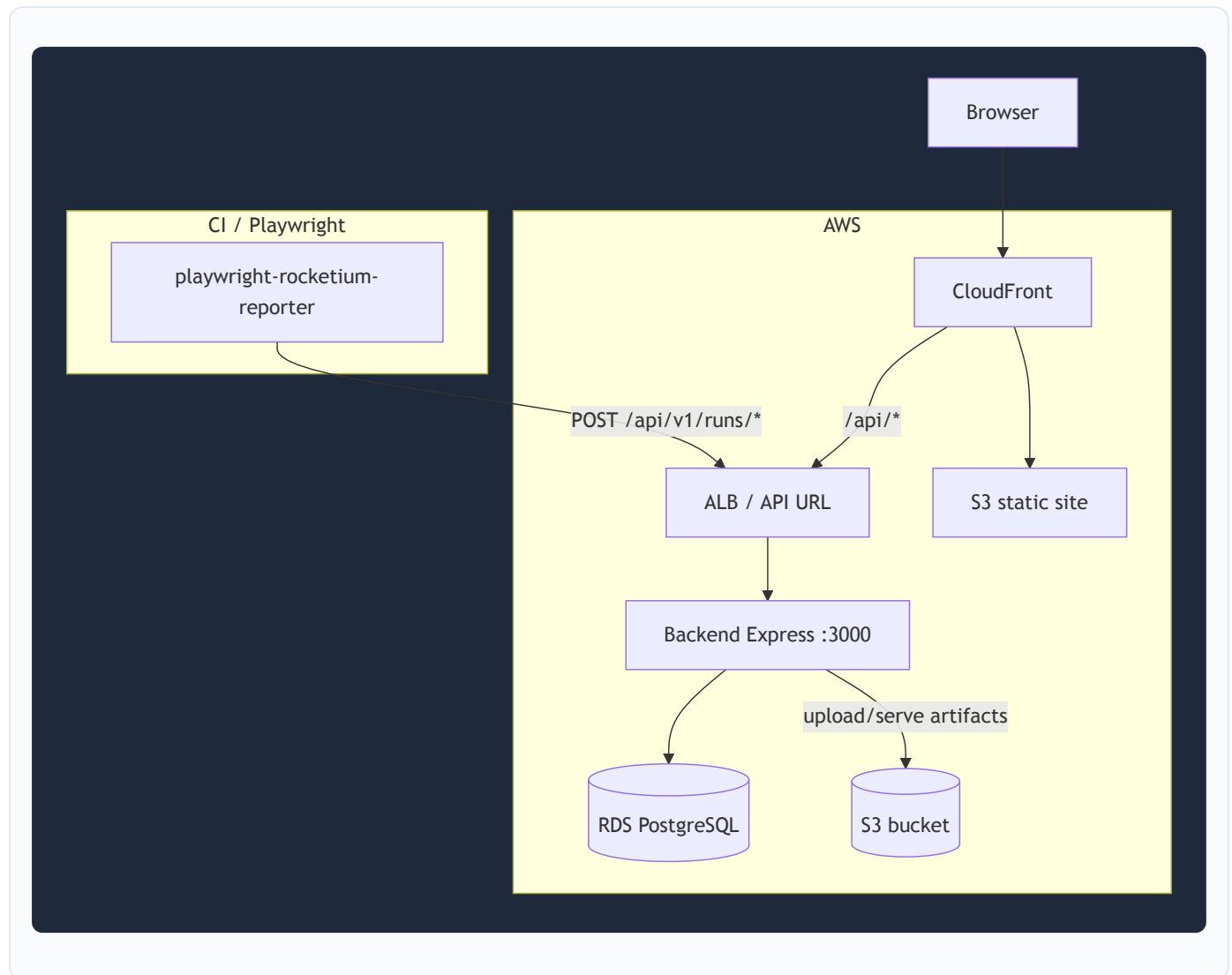
1. Overview

The Playwright Reporter system has four parts:

Component	Repo	Role
Backend API	playwright-reporter-backend	Receives test results, stores metadata in Postgres, files in S3
Dashboard UI	playwright-reporter-ui	React app to view runs, manage projects, get API keys
Reporter package	rocketium-playwright-reporter	Playwright reporter that sends data to the backend
Test suite	automation-tests-2.0	E2E tests — configured to use the reporter in CI

Important: PostgreSQL and S3 work *together*, not as alternatives. RDS stores run metadata; S3 stores binary artifacts (screenshots, videos, traces).

2. Architecture Diagram



3. What Each AWS Service Does

Layer	Stores	Backend env var
RDS PostgreSQL	Runs, shards, specs, tests, artifact pointers, users, projects	<code>DATABASE_URL</code>
S3	Screenshot / video / trace files	<code>ARTIFACT_STORAGE=s3</code> , <code>AWS_S3_BUCKET</code> , <code>AWS_REGION</code>
EC2	Runs Express API (no persistent files when using S3)	<code>PORT</code> , <code>JWT_SECRET</code>
CloudFront + S3	Built React UI (<code>dist/</code>)	None — static files only

Schema is created automatically on backend startup (`CREATE TABLE IF NOT EXISTS` in `src/db/schema.ts`).

4. Step 1 — RDS PostgreSQL

4.1 Create the database

1. AWS Console → **RDS** → **Create database**
2. **Engine:** PostgreSQL **16**
3. **Template:** Free tier (or Dev/Test)
4. **DB instance identifier:** playwright-reporter-db
5. **Master username:** postgres
6. **Master password:** choose a strong password (save it)
7. **DB name:** rocketium_e2e_runs
8. **Instance class:** db.t4g.micro (or larger for production)
9. **Storage:** 20 GB gp3
10. **VPC:** same VPC you will use for EC2
11. **Public access:** No
12. **VPC security group:** create new → e.g. reporter-rds-sg
13. Create database

4.2 Security group (RDS)

Type	Port	Source
PostgreSQL	5432	EC2 security group only

Do **not** open port 5432 to 0.0.0.0/0 .

4.3 Note your connection string

```
postgresql://postgres:YOUR_PASSWORD@<rds-endpoint>.region.rds.amazonaws.com:5432/rocketium_e2e
```

URL-encode special characters in the password: @ → %40 , # → %23 , % → %25

```
postgresql://postgres:MyP%40ss%23word@playwright-reporter-db.xxxx.ap-south-1.rds.amazonaws.com
```

5. Step 2 — S3 Bucket (Artifacts)

5.1 Create bucket

1. AWS Console → **S3** → **Create bucket**
2. **Name:** e.g. rocketium-playwright-artifacts
3. **Region:** same as EC2 (e.g. ap-south-1)

4. Block all public access: ON

5. Create bucket

5.2 IAM policy for EC2

Create an IAM role for the EC2 instance:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::rocketium-playwright-artifacts/artifacts/*"
    },
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::rocketium-playwright-artifacts",
      "Condition": {
        "StringLike": { "s3:prefix": ["artifacts/*"] }
      }
    }
  ]
}
```

Attach this role to EC2 so you **don't need** `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` in `.env`.

5.3 Values to save

Variable	Example
<code>AWS_REGION</code>	<code>ap-south-1</code>
<code>AWS_S3_BUCKET</code>	<code>rocketium-playwright-artifacts</code>
<code>AWS_S3_PREFIX</code>	<code>artifacts</code>

6. Step 3 — EC2 Backend

6.1 Launch EC2

1. **AMI:** Ubuntu 22.04 LTS
2. **Instance type:** `t3.small` minimum
3. **VPC / subnet:** same as RDS
4. **Security group (`reporter-api-sg`):**
 - SSH 22 — your IP
 - HTTP 80 — `0.0.0.0/0` (or ALB SG only)
 - HTTPS 443 — `0.0.0.0/0` (or ALB SG only)
 - TCP 3000 — your IP only (dev; remove after Nginx)
5. Attach IAM role from Step 5.2
6. Launch; note public IP or assign Elastic IP

6.2 Install dependencies on EC2

```
ssh -i your-key.pem ubuntu@<ec2-public-ip>

curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
sudo npm install -g pnpm

git clone <your-repo-url>
cd playwright-reporter-backend
pnpm install
pnpm build
```

6.3 Create backend `.env` on EC2

```
cd playwright-reporter-backend
cp .env.example .env
nano .env
```

Production `.env` example:

```
PORT=3000
```

```
DATABASE_URL=postgresql://postgres:YOUR_ENCODED_PASSWORD@playwright-reporter-db.xxxx.ap-south-
```

```
ARTIFACT_STORAGE=s3
```

```
AWS_REGION=ap-south-1
```

```
AWS_S3_BUCKET=rocketium-playwright-artifacts
```

```
AWS_S3_PREFIX=artifacts
```

```
S3_ARTIFACT_URL_MODE=proxy
```

```
S3_PRESIGN_EXPIRES_SEC=3600
```

```
JWT_SECRET=<generate-a-long-random-string-at-least-32-chars>
```

```
JWT_EXPIRES_IN=7d
```

```
DATA_DIR=./data
```

```
UPLOADS_DIR=./uploads
```

6.4 Start and verify

```
pnpm start  
curl http://localhost:3000/health
```

Expected response:

```
{  
  "ok": true,  
  "service": "playwright-reporter-backend",  
  "storage": "postgres",  
  "artifactStorage": "s3"  
}
```

6.5 Run as a service (PM2)

```
sudo npm install -g pm2  
pm2 start dist/index.js --name reporter-backend  
pm2 save  
pm2 startup
```

6.6 Nginx reverse proxy (recommended)

Point `https://reports-api.yourcompany.com` → `localhost:3000` .

Your **API base URL** for CI and tests:

```
https://reports-api.yourcompany.com
```

7. Step 4 — Frontend (S3 + CloudFront)

7.1 Build the UI

```
cd playwright-reporter-ui
pnpm install
pnpm build
```

Output: `dist/` folder.

7.2 Upload to S3

1. Create bucket: e.g. `rocketium-reporter-ui`
2. Upload all files from `dist/`

7.3 CloudFront distribution

Path pattern	Origin
<code>/api/*</code>	API origin (EC2 / ALB)
<code>/*</code> (default)	S3 UI bucket

Set custom error 403/404 → `/index.html` for SPA routing.

Custom domain: `reports.yourcompany.com` + ACM certificate.

The UI calls `/api/ui/...` on the same domain — no frontend env vars needed in production.

8. Step 5 — Connect Code & Credentials

After AWS infra is up, wire the application code.

8.1 Create project and get API key (Dashboard)

1. Open `https://reports.yourcompany.com`
2. **Sign up / Log in**
3. **Create project** → e.g. "Rocketium Staging"
4. Copy from project settings:
 - **Project ID** — e.g. `9gEjLh`
 - **API Key** — e.g. `rptr_live_f814f717...` (shown once — save immediately)

8.2 Backend — no reporter keys in backend `.env`

The backend validates per-project API keys from the database (created via UI). Backend needs only:

`DATABASE_URL`, S3 vars, `JWT_SECRET`.

8.3 Local testing — automation-tests-2.0/.env

```
REPORTER_API_URL=https://reports-api.yourcompany.com
REPORTER_API_KEY=rptr_live_your_key_here
REPORTER_PROJECT_ID=9gEjLh
REPORTER_TAGS=staging
REPORTER_DEBUG=true
```

Read by automation-tests-2.0/reporter.config.ts :

```
apiUrl: env("REPORTER_API_URL") ?? DEFAULT_REPORTER_API_URL,
apiKey: env("REPORTER_API_KEY"),
projectId: env("REPORTER_PROJECT_ID") || "rocketium-staging",
```

8.4 Playwright config

Reporter runs in CI when REPORTER_API_URL is set (playwright.config.ts):

```
const useRocketiumReporter =
  process.env.CI && !process.env.DISABLE_REPORTER && !!reporterConfig.apiUrl?.trim();
```

For local runs with reporter, set CI=true or adjust that condition.

8.5 GitHub Actions secrets

Repository → **Settings** → **Secrets and variables** → **Actions**:

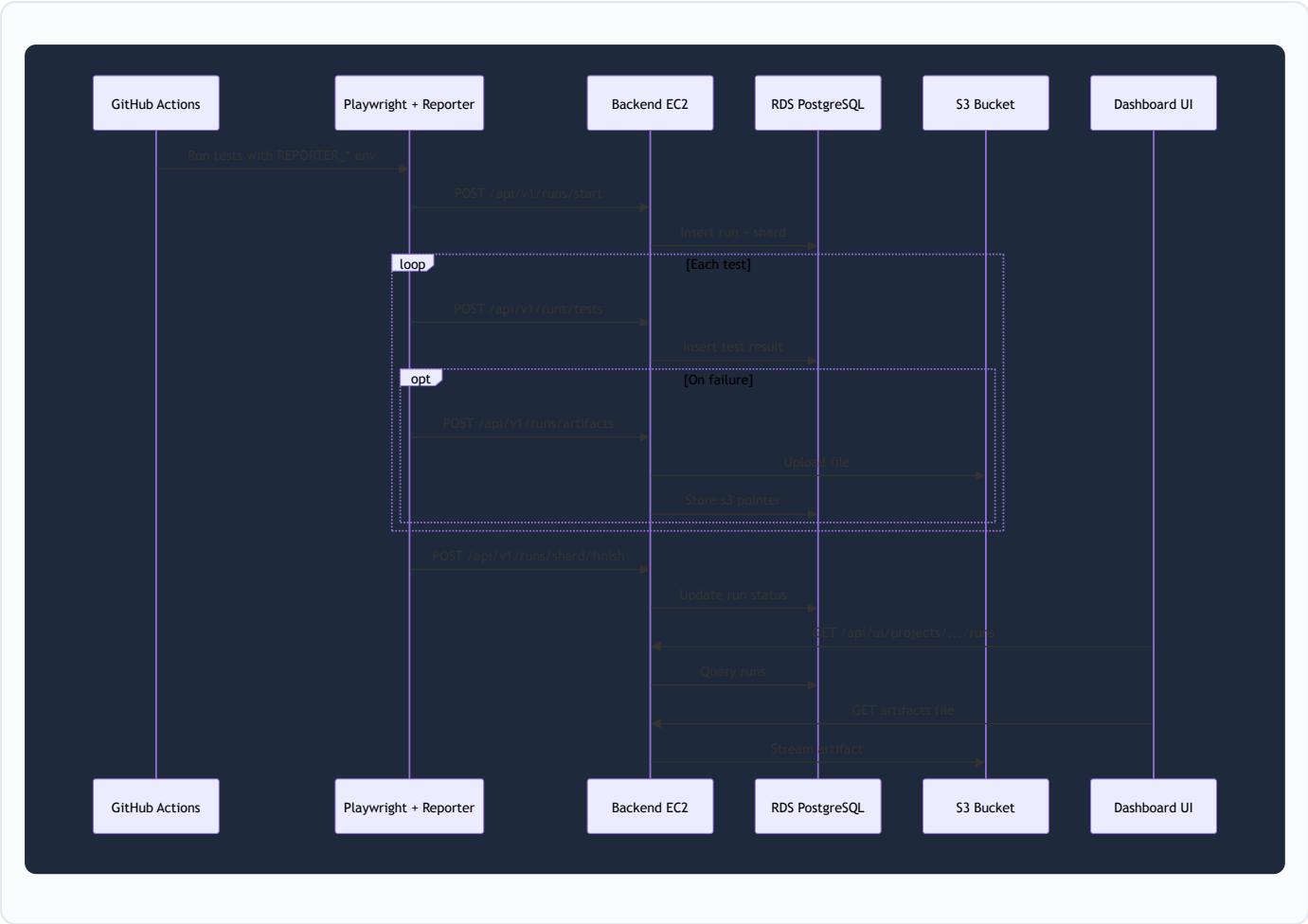
Secret name	Value
REPORTER_API_URL	https://reports-api.yourcompany.com
REPORTER_API_KEY	rptr_live_... from UI

In workflow YAML (e.g. .github/workflows/staging-dynamic.yml):

```
env:
  REPORTER_API_URL: ${ secrets.REPORTER_API_URL }
  REPORTER_API_KEY: ${ secrets.REPORTER_API_KEY }
  REPORTER_PROJECT_ID: 9gEjLh
  REPORTER_CI_BUILD_ID: ${ github.run_id }
  REPORTER_SHARD_NUMBER: ${ matrix.shard }
  REPORTER_EXPECTED_SHARD_COUNT: "12"
  REPORTER_TAGS: staging
```

Variable	Purpose
REPORTER_CI_BUILD_ID	Groups all shards into one run (github.run_id)
REPORTER_SHARD_NUMBER	Shard index (1, 2, 3...)
REPORTER_EXPECTED_SHARD_COUNT	Total shards for the run
REPORTER_TAGS	Labels shown in the dashboard

8.6 Data flow after wiring



9. Step 6 — Verify End-to-End

#	Check	How
1	RDS reachable from EC2	<code>psql \$DATABASE_URL</code> from EC2
2	Backend healthy	<code>curl https://reports-api.../health</code>
3	UI loads	Open <code>https://reports.yourcompany.com</code>
4	Login works	Sign up / log in
5	Project created	Note Project ID + API key
6	CI secrets set	<code>REPORTER_API_URL</code> , <code>REPORTER_API_KEY</code> in GitHub
7	Test run completes	Trigger staging workflow
8	Run appears in UI	Project → Runs list
9	Artifacts viewable	Open failed test → screenshot/video loads

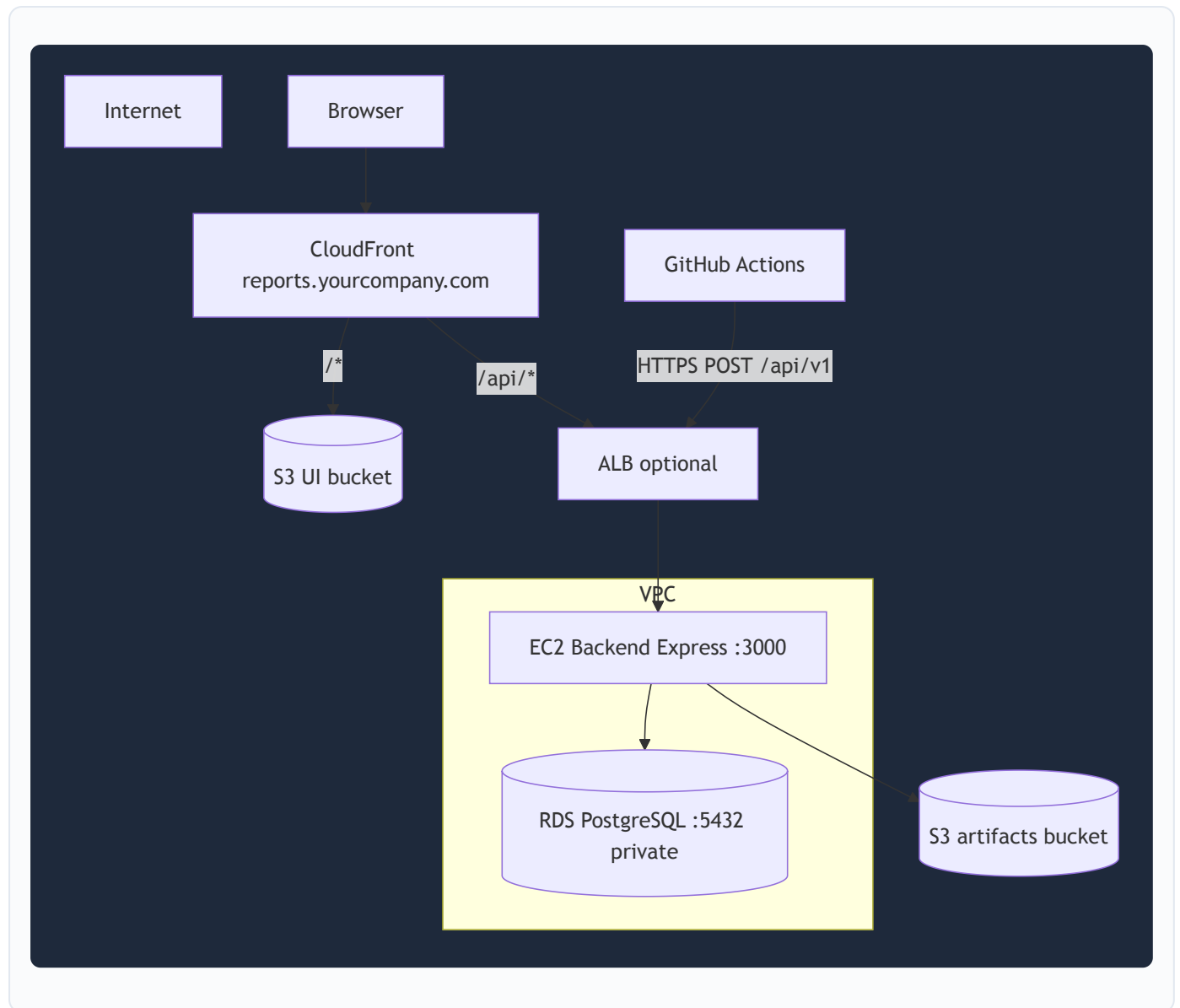
Quick local verification

```
# Terminal 1
cd playwright-reporter-backend && pnpm db:up && pnpm dev

# Terminal 2
cd playwright-reporter-ui && pnpm dev

# Terminal 3
cd automation-tests-2.0
export REPORTER_API_URL=http://localhost:3000
export REPORTER_API_KEY=rprr_live...
export REPORTER_PROJECT_ID=your-project-id
export CI=true
pnpm exec playwright test e2e/path/to/one.spec.ts
```

10. AWS Topology Diagram



11. Environment Variables Reference

Backend (playwright-reporter-backend/.env)

Variable	Required	Example
PORT	No	3000
DATABASE_URL	Yes	postgresql://postgres:...@rds.../rocketium_e2e_runs
ARTIFACT_STORAGE	Yes (prod)	s3
AWS_REGION	Yes (if s3)	ap-south-1
AWS_S3_BUCKET	Yes (if s3)	rocketium-playwright-artifacts
AWS_S3_PREFIX	No	artifacts
S3_ARTIFACT_URL_MODE	No	proxy or redirect
JWT_SECRET	Yes	long random string
JWT_EXPIRES_IN	No	7d

Test suite / CI (automation-tests-2.0)

Variable	Required	Example
REPORTER_API_URL	Yes	https://reports-api.yourcompany.com
REPORTER_API_KEY	Yes	rpтр_live_...
REPORTER_PROJECT_ID	Yes	9gEjLh
REPORTER_CI_BUILD_ID	CI	\${{ github.run_id }}
REPORTER_SHARD_NUMBER	CI sharded	1
REPORTER_EXPECTED_SHARD_COUNT	CI sharded	12
REPORTER_TAGS	No	staging
REPORTER_DEBUG	No	true
DISABLE_REPORTER	No	set to skip reporter

Frontend (playwright-reporter-ui)

No production env vars — API calls use relative /api/ui/... via CloudFront proxy.

12. Troubleshooting

Problem	Likely cause	Fix
DB connection fails	RDS SG blocks EC2	Allow 5432 from EC2 SG
<code>artifactStorage: local</code>	<code>ARTIFACT_STORAGE</code> not set	Set <code>ARTIFACT_STORAGE=s3</code>
Artifacts 404	IAM missing S3 permissions	Attach S3 policy to EC2 role
CI runs not in dashboard	Wrong URL / key / project	Match secrets to UI project
UI can't reach API	CloudFront <code>/api</code> not routed	Add behavior <code>/api/*</code> → API origin
Password auth fails	Special chars in password	URL-encode in <code>DATABASE_URL</code>
Reporter not sending in CI	<code>CI</code> not set or no <code>apiUrl</code>	Set <code>REPORTER_API_URL</code> secret

Quick Reference — Deployment Order

1. RDS PostgreSQL → save `DATABASE_URL`
2. S3 artifacts → save bucket name + IAM role
3. EC2 backend → `.env` + `pnpm start` + Nginx
4. S3 + CloudFront UI → build `dist/`, upload, route `/api`
5. Dashboard → create project, copy API key + project ID
6. GitHub secrets → `REPORTER_API_URL`, `REPORTER_API_KEY`
7. CI workflow → `REPORTER_PROJECT_ID`, shard vars
8. Verify → `/health` + run workflow + check UI

13. Security Groups & Creation Order

13.1 Does S3 need a security group?

No. S3 is not inside your VPC and does not use security groups. Access is controlled by:

- **IAM role** on EC2 (PutObject / GetObject)
- **Bucket policy** (optional)
- **Block public access** (keep ON)

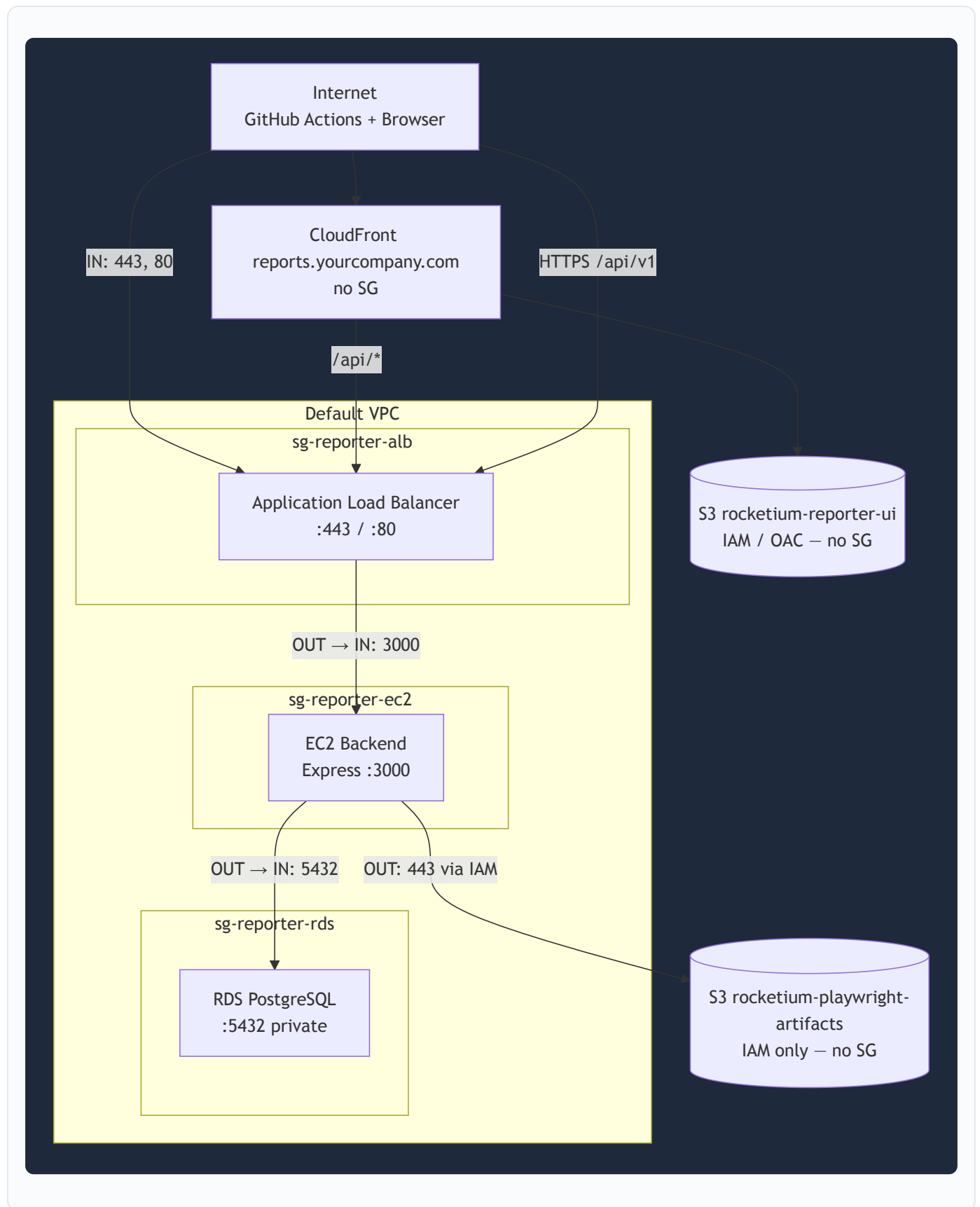
CloudFront also has no VPC security group — it is a global edge service.

13.2 What to create first?

Yes — create security groups before (or while) creating ALB, EC2, and RDS. Recommended order:

1. **IAM role** `reporter-ec2-role` — S3 access (no SG)
2. **S3 buckets** — artifacts + UI static files (no SG)
3. **Security groups** — create all 3 empty shells in the Default VPC:
 - `sg-reporter-alb`
 - `sg-reporter-ec2`
 - `sg-reporter-rds`
4. **RDS** — attach `sg-reporter-rds` , Public access = No
5. **EC2** — attach `sg-reporter-ec2` + IAM role
6. **Add SG rules** — RDS ← EC2, EC2 ← ALB (see tables below)
7. **ALB** — attach `sg-reporter-alb` , target group → EC2:3000
8. **CloudFront** — origins: S3 UI + ALB
9. **Backend .env** + GitHub secrets

13.3 Security group diagram



13.4 Security group rules (named)

sg-reporter-alb

Attached to: **Application Load Balancer**

Direction	Type	Port	Source / Destination	Purpose
Inbound	HTTPS	443	0.0.0.0/0	Browser + GitHub Actions hit API
Inbound	HTTP	80	0.0.0.0/0	Optional — redirect to HTTPS
Outbound	Custom TCP	3000	sg-reporter-ec2	Forward traffic to backend

sg-reporter-ec2

Attached to: **EC2 instance** (playwright-reporter-backend)

Direction	Type	Port	Source / Destination	Purpose
Inbound	Custom TCP	3000	sg-reporter-alb	Only ALB reaches the app
Inbound	SSH	22	YOUR_IP/32	Admin SSH only — not 0.0.0.0/0
Outbound	PostgreSQL	5432	sg-reporter-rds	Backend → database
Outbound	HTTPS	443	0.0.0.0/0	S3 API, package installs, AWS APIs

sg-reporter-rds

Attached to: **RDS PostgreSQL**

Direction	Type	Port	Source / Destination	Purpose
Inbound	PostgreSQL	5432	sg-reporter-ec2	Only backend can query DB
Outbound	All traffic	All	0.0.0.0/0	Default (RDS rarely needs custom outbound)

13.5 S3 & IAM (replaces security group for S3)

Resource	Access control	Who needs access
rocketium-playwright-artifacts	IAM role reporter-ec2-role — s3:PutObject, s3:GetObject on artifacts/*	EC2 backend only
rocketium-reporter-ui	CloudFront OAC + bucket policy (read-only for CloudFront)	CloudFront → public users

13.6 Traffic paths summary

From	To	Path	Controlled by
Browser	UI static files	CloudFront → S3	CloudFront OAC
Browser	API	CloudFront /api/* → ALB → EC2	sg-reporter-alb + sg-reporter-ec2
GitHub Actions	API	HTTPS → ALB → EC2	sg-reporter-alb + sg-reporter-ec2
EC2	RDS	VPC private :5432	sg-reporter-rds + sg-reporter-ec2
EC2	S3 artifacts	HTTPS :443	IAM role (not SG)

13.7 AWS Console — create a security group

1. EC2 → **Security Groups** → **Create security group**
2. Name: e.g. `sg-reporter-rds` , VPC: **Default VPC** (same as RDS/EC2)
3. Add inbound/outbound rules from tables above
4. When referencing another SG, choose **Source type = Security group** and pick the peer SG by name